ソフトウェア工学特論 講義資料

第13回 チーム開発⑤：実装・テスト

- 動作検証の進め方
- テスト設計とCI
- AI活用記録の共有と批判的評価

目次

- 実装の仕上げと動作検証
- テスト設計とCI
- AI活用の批判的評価

実装の仕上げと動作検証

今回の目的と到達目標

今回の目的

- 主要機能を動かし切る
- テストで品質を担保する
- AI活用記録をチームで共有する

到達目標

- [R2-標準] チーム開発ワークフローの完遂
- [R3-標準] プロジェクト管理の遂行

動作検証のチェックリスト

- [ ] **要件ID単位** で動作確認（F-01〜）
- [ ] README の起動手順で **第三者環境で起動可能** か
- [ ] 入力の境界値・異常値で落ちないか
- [ ] エラー時のメッセージが適切か
- [ ] 複数ユーザー同時操作で破綻しないか
- [ ] 本番想定のデータサイズで遅延が許容内か

README の最低要件

```
# プロジェクト名

## 概要
〇〇を管理するWebアプリ

## 使用技術
- Node.js 20, Express
- SQLite

## 起動方法
1. `npm install`
2. `npm run db:init`
3. `npm start`

## メンバー
- 山田 太郎（実装・レビュー）
```

README は **仕様書に貼れる水準** で整備する。

テスト設計とCI

テストの優先順位

種類	目的	優先度
単体テスト	関数・モジュール単位の振る舞い	高（まず書く）
結合テスト	モジュール間の連携	中
E2Eテスト	ユーザー操作の再現	低（時間があれば）

- **まず壊れると困る箇所** にテストを集中させる
- 全網羅より **価値の高い箇所を厚く**
- ルーブリック「理想的」達成にはテスト有りが望ましい

CIの導入例（GitHub Actions）

```yaml
name: test
on: [pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
      - run: npm ci
      - run: npm test
```

- PR時に自動でテスト実行
- 落ちたPRはマージブロック
- **CI = 機械的レビュアー** として運用する

AI活用の批判的評価

AI出力の吟味ポイント

- **事実性**：存在しないAPI・ライブラリを返していないか
- **文脈適合**：自プロジェクトの前提（使用言語・設計方針）に合うか
- **セキュリティ**：SQLインジェクション・XSS対策が抜けていないか
- **保守性**：他メンバーが読んで分かるか
- **出典**：参照元が示されているか（あれば）

「それっぽい」出力を **そのまま採用しない**。

チームでのAI活用記録共有

- 個人の記録を1つのファイル（`ai_log.md`）に集約
- 場面ごとに **プロンプト / 出力 / 判断（採用／修正／却下の理由）**
- 却下したケースも記録する（なぜ採用しなかったかが評価対象）
- 採用率よりも **批判的評価のプロセスの可視化** を評価する

今回のまとめ

- 要件ID単位で動作検証、READMEで第三者起動を保証
- テストはまず壊れると困る箇所に集中、CIで自動化
- AI出力の事実性・文脈・セキュリティを吟味
- AI活用記録は **却下理由も含めて** チームで共有

今回カバーしたMCC項目

- V-D-1 プログラミング
- V-D-4 コンピュータシステム

次回予告

- 第14回: チーム開発⑥ 成果物提出・振り返り